EXPERIMENT-71(HDOC)

C Programming

Program: Student Attendance Management Using a Dynamically Resizable Matrix

Step 1: Understand the Problem Statement

- Input: The initial number of students (1 to 1000) and working days (1 to 31), the attendance of every student for every day (only 1 = Present or 0 = Absent is accepted), and later menu choices such as the number of new students to add.
- Output: The attendance matrix (one row per student, one column per working day), the matrix after new students are added, and the attendance percentage of every student, for example `Student-1: 33.3%`.
- No fixed-size 2D array is used. The whole matrix is one block of integers of `students × days` elements obtained from `malloc()` in `createMatrix()`; element (i, j) is reached with pointer notation as `*(matrix + i * days + j)`.
- When students are added, `addStudents()` enlarges the same block with `realloc()`. The result is first stored in a temporary pointer, so if `realloc()` fails the old matrix is still valid and is kept unchanged. The new rows start as 0 (absent) and are then filled with `recordAttendance()`.
- Separate functions are used for each task: `createMatrix()` (initial matrix), `recordAttendance()` (accepts the attendance), `addStudents()` (adds students), `displayAttendance()` (displays the records), `calculatePercentages()` (percentage of each student) and `releaseMemory()` (frees the memory).
- Invalid input and memory failure are handled: a value other than 0 or 1 (or a non-number) is rejected and asked again, sizes outside the allowed range and invalid menu choices give an error message, and a failed `malloc()` or `realloc()` is reported. Before the program ends, `releaseMemory()` calls `free()` on the matrix. The program was also run under AddressSanitizer (which includes the leak checker) with no errors or leaks reported.

Rule(s) used: Attendance percentage of a student = (number of days present ÷ total working days) × 100, where the number of days present is the sum of that student's row (each present day is 1). Element (i, j) of a matrix stored in one block is at offset i × days + j; adding k students enlarges the block from students × days to (students + k) × days integers.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Students: 3, Days: 3` `Initial attendance entered`	Student 1: 1 0 0 Student 2: 1 0 1 Student 3: 1 1 1	Student 1: 1 0 0 Student 2: 1 0 1 Student 3: 1 1 1	Pass

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
2	`Menu 2: add 3 students (resized to 6 x 3)`	Student 1: 1 0 0 Student 2: 1 0 1 Student 3: 1 1 1 Student 4: 1 0 0 Student 5: 1 1 1 Student 6: 1 0 1	Student 1: 1 0 0 Student 2: 1 0 1 Student 3: 1 1 1 Student 4: 1 0 0 Student 5: 1 1 1 Student 6: 1 0 1	Pass
3	`Menu 3: percentages (6 students)`	Student-1: 33.3% Student-2: 66.7% Student-3: 100% Student-4: 33.3% Student-5: 100% Student-6: 66.7%	Student-1: 33.3% Student-2: 66.7% Student-3: 100% Student-4: 33.3% Student-5: 100% Student-6: 66.7%	Pass
4	`Students: 2, Days: 2` `Entries: x, 2 (invalid), then 1 0 1 1`	Invalid input. Enter 1 (Present) or 0 (Absent). (2 times) Student-1: 50.0% Student-2: 100%	Invalid input. Enter 1 (Present) or 0 (Absent). (2 times) Student-1: 50.0% Student-2: 100%	Pass
5	`Number of students: 0`	Invalid number of students. Enter a value between 1 and 1000.	Invalid number of students. Enter a value between 1 and 1000.	Pass
6	`Students: 3` `Working days: 32`	Invalid number of working days. Enter a value between 1 and 31.	Invalid number of working days. Enter a value between 1 and 31.	Pass
7	`Menu choice: 9`	Invalid choice.	Invalid choice.	Pass
8	`Menu 2: add 0 students`	Invalid number. Enter at least 1 student.	Invalid number. Enter at least 1 student.	Pass
9	`Menu 2: add 1000 students` `(1 + 1000 > 1000)`	Cannot add: total number of students cannot exceed 1000.	Cannot add: total number of students cannot exceed 1000.	Pass
10	`Menu 4: Exit`	Memory released. Exiting program.	Memory released. Exiting program.	Pass

Step 3: Algorithm

1. Start the program.

2. Read the number of students and the number of working days; if either is outside its allowed range, display an error message and stop.

3. Call `createMatrix()` to allocate `students × days` integers with `malloc()`; if it returns NULL, display `Memory allocation failed.` and stop.

4. Call `recordAttendance()`: for every student and day, read the value and ask again until it is 0 or 1, then store it at `*(matrix + i * days + j)`.

5. Repeat until the user chooses Exit: display the menu (1. Display attendance records, 2. Add new students, 3. Calculate attendance percentages, 4. Exit) and read the choice.

6. Choice 1: call `displayAttendance()` to print the matrix with the headings Student, Day 1, Day 2, ...

7. Choice 2: read the number of students to add (at least 1, total not above 1000); call `addStudents()`, which uses `realloc()` to enlarge the block and sets the new rows to 0; then call `recordAttendance()` for the new rows only. If `realloc()` fails, report it and keep the old records.

8. Choice 3: call `calculatePercentages()`, which adds up each row and displays `present × 100 / days` for every student (100% for full attendance, otherwise one decimal place). Choice 4 leaves the loop; any other choice displays `Invalid choice.`

9. Call `releaseMemory()` to `free()` the matrix, set the pointer to NULL and stop the program.

Evaluation against Step 2 test cases: with 3 students and 3 days, the attendance 1 0 0, 1 0 1 and 1 1 1 gives the initial matrix. Adding 3 students enlarges it to 6 rows, and entering 1 0 0, 1 1 1 and 1 0 1 for them keeps the first 3 rows unchanged. The percentages are 1/3 = 33.3%, 2/3 = 66.7%, 3/3 = 100%, 33.3%, 100% and 66.7%, exactly as in the example of the question. For 2 students and 2 days, the entries x and 2 are rejected (2 invalid inputs) before 1 0 1 1 is accepted, giving 50.0% and 100%. A student count of 0, a working-day count of 32, the menu choice 9, adding 0 students and adding 1000 students to a matrix that already has 1 (total 1001, above the limit of 1000) all produce the proper error message — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```
/* HDOC Day-72: Student attendance using a dynamically resizable matrix.

   The attendance matrix has one ROW per student and one COLUMN per working
   day (1 = Present, 0 = Absent). No fixed-size 2D array is used: the whole
   matrix is ONE block of ints obtained with malloc() and enlarged with
   realloc() when students are added. Element (i, j) is reached with pointer
   notation as  *(matrix + i * days + j).  All memory is released with
   free() before the program ends.
*/

#include <stdio.h>
#include <stdlib.h>

#define MAX_STUDENTS 1000
#define MAX_DAYS     31

/* Function prototypes */
int  *createMatrix(int students, int days);
int   recordAttendance(int *matrix, int firstStudent, int lastStudent, int days);
int   addStudents(int **matrix, int *students, int days, int extra);
void  displayAttendance(const int *matrix, int students, int days);
void  calculatePercentages(const int *matrix, int students, int days);
void  releaseMemory(int **matrix);
int   readInt(int *value);
void  clearInputBuffer(void);

int main(void) {
    int *matrix = NULL;
    int students, days, extra, oldStudents, choice, status;

    printf("Enter number of students (1-%d): ", MAX_STUDENTS);
    if (readInt(&students) != 1 || students < 1 || students > MAX_STUDENTS) {
        printf("Invalid number of students. Enter a value between 1 and %d.\n", MAX_STUDENTS);
        return 1;
```

```
    }
    printf("Enter number of working days (1-%d): ", MAX_DAYS);
    if (readInt(&days) != 1 || days < 1 || days > MAX_DAYS) {
        printf("Invalid number of working days. Enter a value between 1 and %d.\n", MAX_DAYS);
        return 1;
    }

    matrix = createMatrix(students, days);       /* malloc() */
    if (matrix == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    if (!recordAttendance(matrix, 0, students, days)) {
        releaseMemory(&matrix);
        printf("\nInput ended. Memory released.\n");
        return 0;
    }

    do {
        printf("\n========== ATTENDANCE MENU ==========\n");
        printf("1. Display attendance records\n");
        printf("2. Add new students\n");
        printf("3. Calculate attendance percentages\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        status = readInt(&choice);
        if (status == -1)
            choice = 4;                          /* input ended: leave safely */
        else if (status == 0)
            choice = 0;                          /* non-numeric input */

        switch (choice) {
            case 1:
                displayAttendance(matrix, students, days);
                break;
            case 2:
                printf("Enter number of students to add: ");
                if (readInt(&extra) != 1 || extra < 1) {
                    printf("Invalid number. Enter at least 1 student.\n");
                    break;
                }
                if (students + extra > MAX_STUDENTS) {
                    printf("Cannot add: total number of students cannot exceed %d.\n", MAX_STUDENTS);
                    break;
                }
                oldStudents = students;
                if (addStudents(&matrix, &students, days, extra)) {      /* realloc() */
                    printf("Matrix resized to %d students x %d days.\n", students, days);
                    if (!recordAttendance(matrix, oldStudents, students, days))
                        choice = 4;              /* input ended: leave safely */
                }
                break;
            case 3:
                calculatePercentages(matrix, students, days);
                break;
            case 4:
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 4);

    releaseMemory(&matrix);                      /* free() */
```

```
    printf("Memory released. Exiting program.\n");
    return 0;
}

/* Creates the initial matrix: students * days integers in one block */
int *createMatrix(int students, int days) {
    return (int *)malloc((size_t)students * days * sizeof(int));
}

/* Records attendance for students firstStudent .. lastStudent-1.
   Only 0 or 1 is accepted. Returns 0 if the input ended, otherwise 1. */
int recordAttendance(int *matrix, int firstStudent, int lastStudent, int days) {
    int i, j, value, status;

    printf("\nEnter attendance (1 = Present, 0 = Absent):\n");
    for (i = firstStudent; i < lastStudent; i++) {
        for (j = 0; j < days; j++) {
            for (;;) {
                printf("Student %d, Day %d: ", i + 1, j + 1);
                status = readInt(&value);
                if (status == -1)
                    return 0;
                if (status == 1 && (value == 0 || value == 1))
                    break;
                printf("Invalid input. Enter 1 (Present) or 0 (Absent).\n");
            }
            *(matrix + i * days + j) = value;
        }
    }
    return 1;
}

/* Adds 'extra' students: grows the block with realloc(). If realloc() fails
   the old matrix is still valid, so it is kept and 0 is returned. */
int addStudents(int **matrix, int *students, int days, int extra) {
    int newTotal = *students + extra;
    int i, j;
    int *temp = (int *)realloc(*matrix, (size_t)newTotal * days * sizeof(int));

    if (temp == NULL) {
        printf("Memory reallocation failed. Existing records are unchanged.\n");
        return 0;
    }
    *matrix = temp;

    for (i = *students; i < newTotal; i++)       /* new rows start as absent */
        for (j = 0; j < days; j++)
            *(*matrix + i * days + j) = 0;

    *students = newTotal;
    return 1;
}

/* Displays the attendance matrix */
void displayAttendance(const int *matrix, int students, int days) {
    int i, j;

    printf("\nAttendance Matrix (1 = Present, 0 = Absent):\n");
    printf("%-12s", "Student");
    for (j = 0; j < days; j++)
        printf("Day %-3d", j + 1);
    printf("\n");
```

```
    for (i = 0; i < students; i++) {
        printf("Student %-4d", i + 1);
        for (j = 0; j < days; j++)
            printf("%-7d", *(matrix + i * days + j));
        printf("\n");
    }
}

/* Calculates and displays the attendance percentage of every student */
void calculatePercentages(const int *matrix, int students, int days) {
    int i, j, present;
    double percentage;

    printf("\nAttendance Percentage:\n");
    for (i = 0; i < students; i++) {
        present = 0;
        for (j = 0; j < days; j++)
            present += *(matrix + i * days + j);
        percentage = present * 100.0 / days;

        if (percentage == 100.0)
            printf("Student-%d: 100%%\n", i + 1);
        else
            printf("Student-%d: %.1f%%\n", i + 1, percentage);
    }
}

/* Releases the matrix and clears the pointer */
void releaseMemory(int **matrix) {
    free(*matrix);
    *matrix = NULL;
}

/* Discards the rest of the current input line */
void clearInputBuffer(void) {
    int ch;
    while ((ch = getchar()) != '\n' && ch != EOF)
        ;
}

/* Reads one integer: 1 = success, 0 = invalid input, -1 = end of input */
int readInt(int *value) {
    int status = scanf("%d", value);
    if (status == EOF)
        return -1;
    if (status != 1) {
        clearInputBuffer();
        return 0;
    }
    return 1;
}
```

sStep 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

cc hdoc72.c	(compile)
./a.out	(execute)

All 10 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 5

```
 Session   Actions    Edit    View    Help

┌──(kali㉿kali)-[~]
└─$ nano hdoc72.c
┌──(kali㉿kali)-[~]
└─$ gcc hdoc72.c
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter number of students (1-1000): 3
Enter number of working days (1-31): 3

Enter attendance (1 = Present, 0 = Absent):
Student 1, Day 1: 1
Student 1, Day 2: 0
Student 1, Day 3: 0
Student 2, Day 1: 1
Student 2, Day 2: 0
Student 2, Day 3: 1
Student 3, Day 1: 1
Student 3, Day 2: 1
Student 3, Day 3: 1

========== ATTENDANCE MENU ==========
1. Display attendance records
2. Add new students
3. Calculate attendance percentages
4. Exit
Enter your choice: 1

Attendance Matrix (1 = Present, 0 = Absent):
Student     Day 1  Day 2  Day 3
Student 1   1      0      0
Student 2   1      0      1
Student 3   1      1      1

========== ATTENDANCE MENU ==========
1. Display attendance records
2. Add new students
3. Calculate attendance percentages
4. Exit
Enter your choice: 2
Enter number of students to add: 3
Matrix resized to 6 students x 3 days.
```

Screenshot 2 of 5

```
 Session   Actions    Edit    View    Help

Enter attendance (1 = Present, 0 = Absent):
Student 4, Day 1: 1
Student 4, Day 2: 0
Student 4, Day 3: 0
Student 5, Day 1: 1
Student 5, Day 2: 1
Student 5, Day 3: 1
Student 6, Day 1: 1
Student 6, Day 2: 0
Student 6, Day 3: 1

========== ATTENDANCE MENU ==========
1. Display attendance records
2. Add new students
3. Calculate attendance percentages
4. Exit
Enter your choice: 1

Attendance Matrix (1 = Present, 0 = Absent):
Student     Day 1  Day 2  Day 3
Student 1   1      0      0
Student 2   1      0      1
Student 3   1      1      1
Student 4   1      0      0
Student 5   1      1      1
Student 6   1      0      1

========== ATTENDANCE MENU ==========
1. Display attendance records
2. Add new students
3. Calculate attendance percentages
4. Exit
Enter your choice: 3

Attendance Percentage:
Student-1: 33.3%
Student-2: 66.7%
Student-3: 100%
Student-4: 33.3%
Student-5: 100%
Student-6: 66.7%
```

Screenshot 3 of 5

```
 Session   Actions    Edit    View    Help

========== ATTENDANCE MENU ==========
1. Display attendance records
2. Add new students
3. Calculate attendance percentages
4. Exit
Enter your choice: 4
Memory released. Exiting program.
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter number of students (1-1000): 2
Enter number of working days (1-31): 2

Enter attendance (1 = Present, 0 = Absent):
Student 1, Day 1: x
Invalid input. Enter 1 (Present) or 0 (Absent).
Student 1, Day 1: 2
Invalid input. Enter 1 (Present) or 0 (Absent).
Student 1, Day 1: 1
Student 1, Day 2: 0
Student 2, Day 1: 1
Student 2, Day 2: 1

========== ATTENDANCE MENU ==========
1. Display attendance records
2. Add new students
3. Calculate attendance percentages
4. Exit
Enter your choice: 3

Attendance Percentage:
Student-1: 50.0%
Student-2: 100%

========== ATTENDANCE MENU ==========
1. Display attendance records
2. Add new students
3. Calculate attendance percentages
4. Exit
Enter your choice: 4
Memory released. Exiting program.
```

Screenshot 4 of 5

```
 Session   Actions    Edit    View    Help

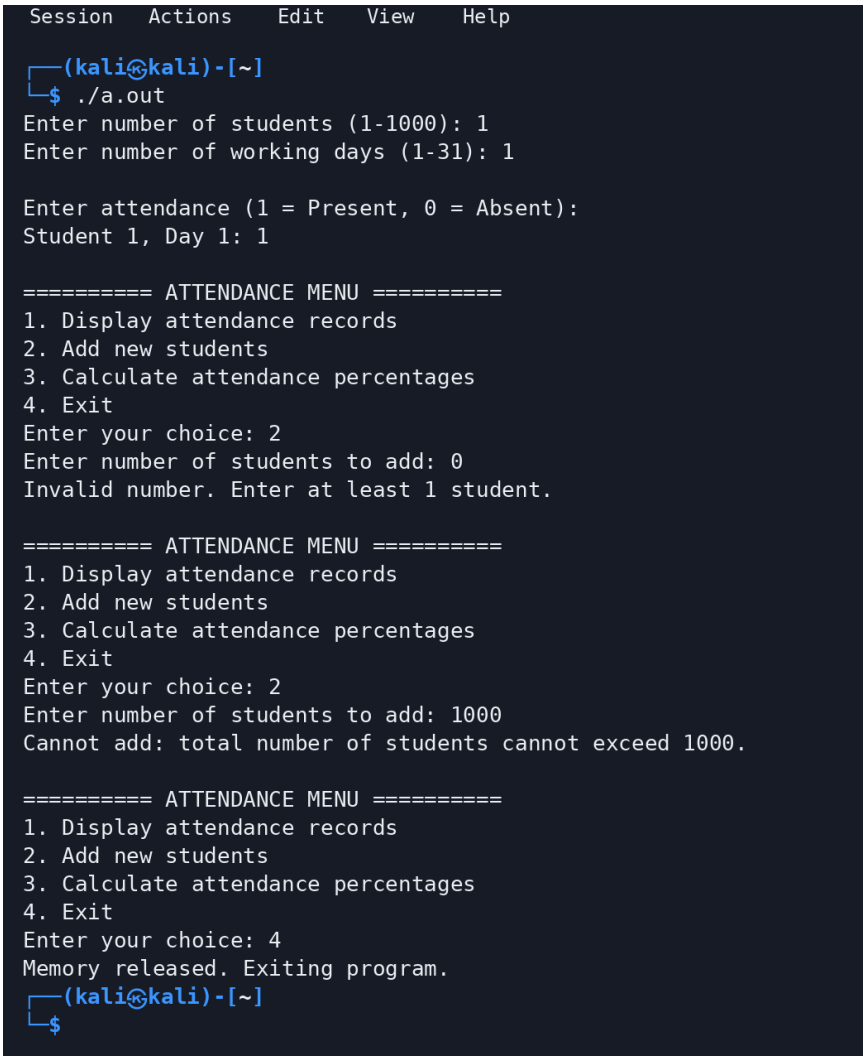
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter number of students (1-1000): 0
Invalid number of students. Enter a value between 1 and 1000.
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter number of students (1-1000): 3
Enter number of working days (1-31): 32
Invalid number of working days. Enter a value between 1 and 31.
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter number of students (1-1000): 1
Enter number of working days (1-31): 1

Enter attendance (1 = Present, 0 = Absent):
Student 1, Day 1: 1

========== ATTENDANCE MENU ==========
1. Display attendance records
2. Add new students
3. Calculate attendance percentages
4. Exit
Enter your choice: 9
Invalid choice.

========== ATTENDANCE MENU ==========
1. Display attendance records
2. Add new students
3. Calculate attendance percentages
4. Exit
Enter your choice: 4
Memory released. Exiting program.
```

Screenshot 5 of 5



Conclusion

In this experiment, a student attendance system was built around a dynamically resizable matrix stored as one block of memory: it is created with malloc(), enlarged with realloc() whenever students are added, and accessed through pointer notation instead of a fixed-size 2D array. Separate functions create the matrix, record attendance (accepting only 0 or 1), add students, display the records, calculate each student's attendance percentage and release the memory with free(), while invalid input and allocation failure are handled. The program was compiled with gcc and tested on ten cases, including the exact example from the question (33.3%, 66.7%, 100%, 33.3%, 100%, 66.7%), with every result matching and an AddressSanitizer run showing no memory errors or leaks.